Model Performance Evaluation Framework

Project Cookbook

Team members: Sakshi Manjrekar, Saniya Shaikh, Shruti Bhandari, Vinay BG

Course: M.Sc. Applied Data Science and AI

University: SRH Heidelberg

Supervisor: Prof. Dr. Swati Chandna

Client contacts: Ashwani-Kumar Sadana (Deutsche Bank), Michael-C Meier (Deutsche Bank)

Date: 27-09-2026

Table of Contents

1. Introduction **6**
- 1.1 Purpose of the Cookbook 6
- 1.2 Project Overview 6
- 1.3 Problem Statement 7
- 1.4 Objectives 8
- 1.5 Target Users 8
- 1.6 Scope 9
- 1.7 Current System Architecture 10
- 1.8 Historical Project Development 10
- 1.9 How to Use This Cookbook 11

2. System Overview **11**
- 2.1 Technology Stack 12
- 2.2 End-to-End Workflow 13
- 2.3 Project Structure 14
- 2.4 File Summary 16
- 2.5 Data Flow 17
- 2.6 Security and Configuration Principles 17

3. Environment and Initial Setup **17**
- Recipe 1: Setting Up the Development Environment 17
- Recipe 2: Configuring Environment Variables 18
- Recipe 3: Running the Application 19
- Recipe 4: Running the Harmonizer 19

4. Data Handling **20**
- Recipe 5: Providing the Input Dataset 20
- Recipe 6: Inspecting the Dataset 20
- Recipe 7: Understanding Question Numbers 21
- Recipe 8: Understanding the Harmonization Dictionaries 21
- Recipe 9: Preparing the Harmonized Dataset 22

5. Financial Terminology Harmonization **22**
- Recipe 10: Loading Harmonization Dictionaries 22
- Recipe 11: Building the Alias Map 22
- Recipe 12: Applying Canonical Terminology 23
- Recipe 13: Handling Longest-Match Aliases 23
- Recipe 14: Generating the Harmonization QA Report 23
- Recipe 15: Validating Harmonization Results 24

6. Similar Question Search **25**
- Recipe 16: Preparing the Search Dataset 25
- Recipe 17: Harmonizing a New Question 25
- Recipe 18: Preparing the Gemini Prompt 25
- Recipe 19: Running the Gemini Search 26

Recipe 20: Validating the Gemini Response 26
Recipe 21: Ranking and Limiting Results 27
Recipe 22: Recovering the Original Historical Question 27
7. User Interface 27
Recipe 23: Starting the Dash Application 27
Recipe 24: Evaluator Identification 28
Recipe 25: Entering a Search Question 28
Recipe 26: Understanding Search Results 28
Recipe 27: Understanding Match-Strength Labels 28
Recipe 28: Handling UI Errors 29
8. Gemini and OpenRouter Integration 29
Recipe 29: Configuring the Gemini Model 29
Recipe 30: OpenRouter Communication 29
Recipe 31: Structured JSON Responses 30
Recipe 32: Rate Limiting 30
Recipe 33: Retry and Backoff 30
Recipe 34: API Error Handling 31
9. Testing and Quality Assurance 31
Recipe 35: Running the Test Suite 31
Recipe 36: Harmonizer Tests 31
Recipe 37: Gemini Client Tests 32
Recipe 38: Similarity Search Tests 32
Recipe 39: Linting 32
Recipe 40: Continuous Integration 33
10. Logging and Auditability 33
Recipe 41: Application Logging 33
Recipe 42: Search Audit Logging 33
Recipe 43: Reviewing Search Logs 34
11. Outputs and Results 34
Recipe 44: Harmonized Dataset Output 34
Recipe 45: Harmonization QA Output 34
Recipe 46: Search Results 35
Recipe 47: Result Interpretation 35
12. Reproducibility 35
Recipe 48: Reproducing the Development Environment 35
Recipe 49: Reproducing a Harmonization Run 35
Recipe 50: Reproducing a Search 36
Recipe 51: Using a New Dataset 36
13. Deployment 36
Recipe 52: Building the Docker Image 36
Recipe 53: Running the Docker Container 37
Recipe 54: GitHub Container Registry 37

Recipe 55: Portainer Deployment 37
Recipe 56: Persistent Logs and Outputs 38
14. Troubleshooting 38
14.1 Installation Problems 38
14.2 Dataset Problems 38
14.3 Harmonization Problems 39
14.4 Gemini/API Problems 39
14.5 UI Problems 39
14.6 Docker Problems 39
14.7 Logging Problems 39
15. Best Practices 40
15.1 Code Organization 40
15.2 Data Management 40
15.3 Harmonization 40
15.4 LLM Usage 40
15.5 Security 41
15.6 Testing 41
15.7 Reproducibility 41
15.8 Deployment 41
16. Complete Example 41
16.1 Example Input 41
16.2 Harmonization 41
16.3 Search Execution 42
16.4 Gemini Response 42
16.5 Result Processing 42
16.6 UI Output 42
16.7 Audit Record 43
17. Project Evolution 43
17.1 First Iteration: Similar Question Search 44
17.2 Second Iteration: Similar Question Search 44
17.3 Third Iteration: Model Performance Evaluation Framework 44
17.4 Fourth Iteration: Question Similarity 45
17.5 Fifth Iteration: Question Similarity with Dash Mantine 45
17.6 Sixth Iteration: Model Performance Evaluation Framework 45
17.7 Seventh Iteration: Model Performance Evaluation Framework Final 46
17.8 Eighth Iteration: Model Performance Evaluation Framework Final 46
17.9 Transition to the Current Similar Question Search System 47
17.10 Evolution of the User Interface 48
17.11 Evolution of the Matching Strategy 49
17.12 Evolution of Reliability and Quality Assurance 49
17.13 Overall Project Evolution 49
18. Appendix 50
A. Installation Commands 50

B. Main Configuration Values 51
C. Match-Strength Bands 51
D. Dataset Schema 51
E. Harmonization Dictionary Schema 52
F. Gemini Response Schema 52
G. Known Acceptance Test Cases 52
H. Docker Configuration 53
I. Error Reference 53
J. Glossary 54
K. Known Limitations 54
L. References 55

1. Introduction

1.1 Purpose of the Cookbook

This cookbook provides the operational documentation for the Similar Question Search component of the GenAI Model Performance Evaluation Framework.

The document is intended to allow a new team member, reviewer, developer, data scientist, or future maintainer to understand the system without having to reverse-engineer the source code.

The cookbook explains:

- how to install the project
- how to configure the environment
- how the historical financial dataset is structured
- how financial terminology is harmonized
- how a new question is processed
- how Gemini performs semantic comparison
- how search results are validated and displayed
- how audit logging works
- how the automated tests are executed
- how the application is deployed
- how common problems can be diagnosed
- and how the system can be extended

The documentation follows a recipe-based structure. Each recipe describes the problem or use case, prerequisites, inputs, method, implementation, architecture, flow, outputs, validation, troubleshooting, best practices, variations, and references.

1.2 Project Overview

The Similar Question Search component addresses a practical problem in financial model evaluation. Evaluators work with a growing historical collection of financial-analytics questions. A newly created question may already have been asked in an earlier evaluation, but the wording may be different. For example, a historical question may use:

- ROE
- equity return
- shareholder return
- Return on Equity

while another question may describe the same financial concept using different terminology. Similarly, divisions may appear using abbreviations or alternative names such as:

- CB
- Corporate Banking
- Corp Bank

Or:

- WM
- Wealth Management
- Wealth Mgmt.

Reporting periods may also appear in different forms, such as:

- Q1 2026
- 1Q26
- first quarter 2026
- January to March 2026

A simple keyword search would not reliably identify all of these as related questions. The Similar Question Search system therefore combines deterministic financial terminology harmonization with semantic comparison performed by Gemini. The evaluator enters a question into a browser-based application. The question is harmonized using the same terminology dictionaries used for the historical dataset. The harmonized question and the harmonized historical question dataset are then provided to Gemini through OpenRouter.

Gemini evaluates the historical questions and returns the questions that genuinely ask for the same underlying analysis. The application displays the results using:

- conversation ID
- question number
- original historical question text
- similarity score
- plain-language match strength
- and an explanation of why the questions match

The system also records every search in an audit log.

1.3 Problem Statement

Financial question datasets frequently contain semantically equivalent questions that differ in wording, abbreviations, terminology, division names, and reporting-period expressions. Traditional lexical matching is insufficient because two questions can describe the same analytical task without sharing the same exact words. For example: “What is the ROE for Corporate Banking?”, and “How did Corporate Banking perform in terms of equity return?” can describe the same underlying financial analysis.

The problem is therefore to identify historically repeated or semantically similar financial questions while preserving enough information to distinguish questions that only appear similar. The system addresses this problem through two main mechanisms:

1. Deterministic terminology harmonization
2. LLM-based semantic similarity evaluation

The harmonization stage provides a consistent vocabulary. The Gemini stage determines whether two questions are substantively asking the same thing. The system therefore does not depend exclusively on word overlap.

1.4 Objectives

The main objectives of the Similar Question Search component are:

1. Provide a simple interface for entering a new financial question.
2. Normalize inconsistent financial terminology.
3. Normalize abbreviations and alternative expressions.
4. Normalize reporting-period terminology.
5. Apply the same harmonization process to historical and new questions.
6. Search the historical question dataset for semantically equivalent questions.
7. Return every genuine match identified by the model.
8. Provide a similarity score between 0 and 1.
9. Provide a short explanation for each match.
10. Preserve the exact historical question text for traceability.
11. Identify the exact conversation and question number associated with each match.
12. Provide an evaluator-facing interface that does not require technical knowledge.
13. Maintain an audit trail of searches.
14. Handle API failures and rate limits gracefully.
15. Provide automated unit tests for the core functionality.
16. Support containerized deployment.

1.5 Target Users

#	User Group	How They Use the System
1	Financial Analysts	Financial analysts can use the application to identify whether a financial question has already been asked.
2	Data Scientists	Data scientists can use the project to study semantic similarity, financial terminology normalization, and LLM-based matching.

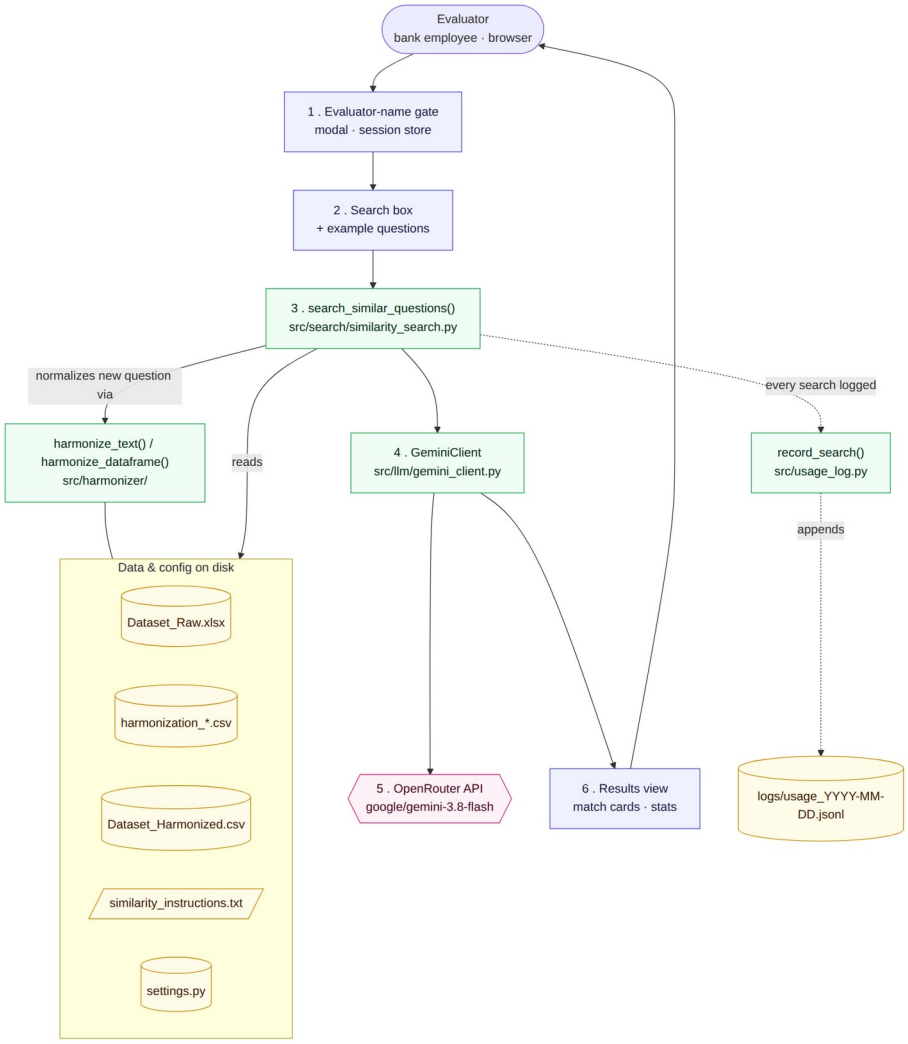
#	User Group	How They Use the System
3	AI/ML Engineers	AI/ML engineers can maintain the harmonization pipeline, Gemini integration, search logic, testing framework, and deployment infrastructure.
4	Financial Institutions	Financial institutions can use the approach to organize large historical collections of financial questions and reduce redundant evaluation effort.
5	Researchers and Students	Researchers and students can use the system as a practical example of applying LLMs and deterministic NLP preprocessing to financial text.

1.6 Scope

The current scope includes:

1. Data Preparation: loading the historical Excel dataset; inspecting the dataset; assigning question numbers; harmonizing financial terminology; producing a clean CSV representation.
2. Similar Question Search: harmonizing the evaluator's new question; preparing the historical dataset for the model; sending the new question and historical dataset to Gemini; receiving structured similarity results; validating the response; sorting and limiting results.
3. User Interface: evaluator-name gate; search interface; example questions; search results; match-strength labels; error notifications; light/dark mode.
4. Reliability: rate limiting; retry and exponential backoff; JSON response validation; audit logging; unit tests; CI checks.
5. Deployment: Docker; GitHub Actions; GitHub Container Registry; Portainer deployment.

1.7 Current System Architecture



End-to-end architecture of the Similar Question Search component

1.8 Historical Project Development

The historical project files contain eight development iterations:

1. Similar Question Search, 1st iteration
2. Similar Question Search, 2nd iteration
3. Model Performance Evaluation Framework, 3rd iteration
4. Question Similarity, 4th iteration
5. Question Similarity Dash Mantine, 5th iteration
6. Model Performance Evaluation Framework, 6th iteration
7. Model Performance Evaluation Framework Final, 7th iteration

8. Model Performance Evaluation Framework Final, 8th iteration

The historical cookbook documents an earlier architecture involving:

```
Harmonization
↓
Candidate Retrieval
↓
Deterministic Checks
↓
Gemini Verification
↓
Report Cross-Check
↓
Similarity Graph
↓
Duplicate Groups
```

That architecture should be understood as part of the project's development history. The current Similar Question Search component represents the later simplified operational architecture.

1.9 How to Use This Cookbook

A new developer should read the cookbook in the following order:

1. Introduction
2. System Overview
3. Environment and Initial Setup
4. Data Handling
5. Harmonization
6. Similar Question Search
7. Testing
8. Logging
9. Deployment
10. Troubleshooting

A user who only needs to run the application can follow Recipes 1 to 5 and Recipe 24 onward.

A developer modifying similarity behavior should additionally read Recipes 18 to 23 and Appendix F.

A developer modifying harmonization should read Recipes 11 to 16.

A deployment administrator should read Chapter 13.

2. System Overview

2.1 Technology Stack

#	Component	Technology
1	Programming Language	Python 3.11+
2	User Interface	Dash
3	UI Components	Dash Mantine Components
4	LLM	Google Gemini 3.8 Flash
5	Model Gateway	OpenRouter
6	Data Processing	pandas
7	Excel Processing	openpyxl
8	HTTP Fallback	requests
9	API Client	OpenAI-compatible Python SDK
10	Configuration	python-dotenv
11	Testing	pytest
12	Test Mocking	pytest-mock
13	Coverage	pytest-cov
14	Linting	ruff
15	Packaging	pyproject.toml
16	Containerization	Docker
17	CI/CD	GitHub Actions
18	Container Registry	GitHub Container Registry
19	Deployment	Portainer

2.2 End-to-End Workflow

1. Load raw historical dataset
↓
2. Load entity and time dictionaries
↓
3. Build alias-to-canonical mappings
↓
4. Compile regular-expression patterns
↓
5. Assign Question_Number values
↓
6. Harmonize historical conversation text
↓
7. Write Dataset_Harmonized.csv
↓
8. Evaluator enters a new question
↓
9. New question is harmonized
↓
10. Historical rows are formatted for Gemini
↓
11. New question and historical dataset sent to Gemini
↓
12. Gemini identifies genuine matches
↓
13. Gemini returns structured JSON
↓
14. Response is parsed and validated
↓
15. Results are sorted by similarity
↓
16. Maximum display count is applied
↓
17. Original historical text is recovered
↓
18. Results displayed in Dash
↓
19. Search written to audit log

2.3 Project Structure

```
model-performance-evaluation-framework-final/
├── app.py
├── README.md
├── Dockerfile
├── .dockerignore
├── .env.example
├── .gitignore
├── architecture.mmd
├── architecture_diagram.png
├── conftest.py
├── pyproject.toml
├── requirements.txt
│
├── config/
│   ├── __init__.py
│   └── settings.py
│
├── data/
│   ├── Dataset_Raw.xlsx
│   ├── harmonization_entities.csv
│   └── harmonization_time_formats.csv
│
├── instructions/
│   └── similarity_instructions.txt
│
├── outputs/
│   ├── Dataset_Harmonized.csv
│   └── dataset_harmonization_qa_report.txt
│
├── logs/
│   └── usage_YYYY-MM-DD.jsonl
│
├── src/
│   ├── __init__.py
│   ├── logging_config.py
│   ├── usage_log.py
│   │
│   ├── harmonizer/
│   │   ├── __init__.py
│   │   └── harmonizer.py
│   │
│   ├── llm/
│   │   ├── __init__.py
│   │   └── gemini_client.py
│   │
│   └── search/
```

```
│       ├── __init__.py
│       └── similarity_search.py
│
└── tests/
    ├── test_harmonizer.py
    ├── test_gemini_client.py
    └── test_similarity_search.py
```

2.4 File Summary

Path	Purpose
app.py	Dash UI and user interaction
config/settings.py	Central configuration
data/Dataset_Raw.xlsx	Historical raw dataset
data/harmonization_entities.csv	Entity/terminology mappings
data/harmonization_time_formats.csv	Time/date mappings
instructions/similarity_instructions.txt	Gemini similarity rules
outputs/Dataset_Harmonized.csv	Harmonized dataset
outputs/ dataset_harmonization_qa_report.txt	Harmonization QA report
src/harmonizer/harmonizer.py	Harmonization functions
src/search/similarity_search.py	Search orchestration
src/llm/gemini_client.py	OpenRouter/Gemini integration
src/usage_log.py	Search audit logging
src/logging_config.py	Application logging
tests/test_harmonizer.py	Harmonizer tests
tests/test_gemini_client.py	Gemini client tests
tests/test_similarity_search.py	Search pipeline tests
Dockerfile	Container build/deployment
.github/workflows/ci.yml	Automated testing and Docker publishing
pyproject.toml	Python project metadata and development dependencies
requirements.txt	Runtime dependencies

OpenRouter provides access to google/gemini-3.8-flash. The model is therefore not called directly through Google AI Studio.

2.5 Data Flow

There are three main data representations.

- Raw dataset: data/Dataset_Raw.xlsx
- Harmonized dataset: outputs/Dataset_Harmonized.csv
- Gemini Search Representation: each relevant row is represented using a label containing [Conversation_ID Question_Number] followed by the harmonized question text.

2.6 Security and Configuration Principles

The project uses environment variables for secrets. The OpenRouter API key must never be committed to source control.

The .env file is intended for local development and is ignored by Git.

For Docker/Portainer deployment, the key should be supplied as an environment variable rather than being included in the Docker image. The project also avoids hardcoding deployment-specific values wherever possible.

3. Environment and Initial Setup

Recipe 1: Setting Up the Development Environment

1. Problem / Use Case

A new developer needs to install the project and create an isolated environment in which the application and its tests can run.

2. Prerequisites

- Python 3.11 or newer
- Git
- pip
- internet access
- an OpenRouter API key for live searches

3. Input

The project repository.

4. Method

The recommended setup is: clone the repository, create a virtual environment, activate the virtual environment, install dependencies, configure .env, run tests, then start the application.

5. Implementation

```
git clone <repository-url>
cd model-performance-evaluation-framework-final

python -m venv .venv

Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate

pip install -e ".[dev]"
```

Alternatively:

```
pip install -r requirements.txt
```

6. Validation / Testing

Run pytest -q. The current project test suite contains 23 tests, and all 23 currently pass, covering harmonization, Gemini response parsing, retry behavior, and similarity-search result processing.

7. Troubleshooting

If a package cannot be imported, reinstall the project dependencies.

8. Best Practices

Use a virtual environment so that project dependencies remain isolated from other Python projects.

9. References

- requirements.txt
- README.md

Recipe 2: Configuring Environment Variables

1. Problem / Use Case

The application requires an API key to communicate with OpenRouter.

2. Prerequisites

- Recipe 1

3. Input

The project contains: .env.example

4. Method / Approach

Create a .env file from the example configuration.

5. Implementation

```
cp .env.example .env
```

On Windows, the file can also be copied manually.

Set:

```
GEMINI_API_KEY=YOUR_OPENROUTER_KEY
```

Despite the variable name, this value is an OpenRouter API key.

6. Additional Settings

The project supports:

```
DASH_HOST=127.0.0.1
DASH_PORT=8050
DASH_DEBUG=0
DASH_NO_BROWSER=0
LOG_LEVEL=INFO
```

7. Output

The application loads the configuration through python-dotenv.

8. Validation / Testing

The Gemini client raises an error if no API key is available.

9. Best Practices

Never commit the .env file or the real API key to GitHub. The .env file is intended to remain git-ignored.

Recipe 3: Running the Application

From the project root, run python app.py. The default local address is http://127.0.0.1:8050.

The application may automatically open a browser window.

To disable automatic browser opening, set DASH_NO_BROWSER=1.

For debugging, set DASH_DEBUG=1.

Recipe 4: Running the Harmonizer

The harmonizer can be run independently:

```
python -c "from src.harmonizer import run_harmonization_raw; run_harmonization_raw()"
```

It reads data/Dataset_Raw.xlsx and generates outputs/Dataset_Harmonized.csv and outputs/dataset_harmonization_qa_report.txt.

The harmonizer is also automatically run when the application starts and the harmonized output does not yet exist.

4. Data Handling

Recipe 5: Providing the Input Dataset

1. Input

The current raw dataset is: data/Dataset_Raw.xlsx

The current project dataset contains:

- 637 total rows
- 334 user rows
- 303 model rows
- 117 unique conversation IDs

The raw dataset contains four columns:

Column	Description
Conversation_ID	Identifier of the conversation
Date	Date associated with the row
Type	user or model
Conversation	Full turn text

The raw dataset must not be overwritten by processing. The harmonizer writes a separate CSV output.

Recipe 6: Inspecting the Dataset

Use pandas:

```
df = pd.read_excel("data/Dataset_Raw.xlsx")

print(df.shape)
print(df.columns.tolist())
print(df["Type"].value_counts())
print(df.isna().sum())
print(df["Conversation_ID"].nunique())
```

The current dataset produces:

- Shape: (637, 4)
- Column names: ['Conversation_ID', 'Date', 'Type', 'Conversation']
- Type distribution: user = 334 and model = 303
- Unique conversations: 117

There are 33 missing values in the Conversation column in the current raw dataset. Missing values must not automatically be interpreted as an error because the dataset contains different types of conversation records.

Recipe 7: Understanding Question Numbers

The current harmonizer creates: Question_Number

Only user rows receive question numbers. Model rows receive an empty value. Question numbers are assigned according to the order of user questions within each conversation.

For example: user leads to Q1, then model, then user leads to Q2, then model, then user leads to Q3, then a second consecutive user row stays Q3, then model.

Two consecutive user rows with the same Conversation ID are treated as a split question. Therefore, [C69 Q3] may correspond to two consecutive source rows. The second row is not treated as a separate question. This makes the question label precise: [Conversation_ID Question_Number]. For example, [C42 Q3] identifies Question 3 of Conversation C42.

Recipe 8: Understanding the Harmonization Dictionaries

The project uses two dictionaries.

Entity Dictionary: data/harmonization_entities.csv

Current size: 423 rows

It contains nine active categories:

- FINANCIAL_TERM
- DIVISION
- REGION
- ACTION
- REGULATOR
- RATING_AGENCY
- COMPANY
- STRATEGY
- EXECUTIVE

Time Dictionary: data/harmonization_time_formats.csv

Current size: 204 rows

It contains time-related categories including:

- RELATIVE_TIME
- MONTH_YEAR
- DATE

- QUARTER_YEAR
- NUMBER_FORMAT
- COMPARISON
- MONTH
- FULL_YEAR
- HALF_YEAR
- QUARTER
- NINE_MONTH_YEAR

The two dictionaries currently contain 627 rows in total: 423 entity/terminology rows and 204 time-format rows. The 24 NUMBER_FORMAT rows are loaded but excluded from text substitution, leaving 603 dictionary rows eligible for harmonization.

Recipe 9: Preparing the Harmonized Dataset

The complete process is: load Dataset_Raw.xlsx, load the entity dictionary, load the time dictionary, combine dictionaries, build the alias map, compile regex patterns, load the raw dataset, assign Question_Number, harmonize the Conversation column, write Dataset_Harmonized.csv, and write the QA report.

The output schema contains:

- Conversation_ID
- Date
- Type
- Question_Number
- Conversation
- Conversation_harmonized

The current harmonized output contains 637 rows and 6 columns.

5. Financial Terminology Harmonization

Recipe 10: Loading Harmonization Dictionaries

The harmonizer reads both CSV dictionaries as strings. This is intentional because values such as 2025, Q1, and Q2 2025 must not be accidentally converted into numeric or date objects. The dictionaries are combined into a single pandas DataFrame.

Recipe 11: Building the Alias Map

The raw dictionary contains canonical terms and multiple aliases. The harmonizer converts the dictionary into alias to canonical pairs. For example:

```
roe -> Return on Equity
equity return -> Return on Equity
shareholder return -> Return on Equity

cir -> Cost-Income Ratio
cost income ratio -> Cost-Income Ratio
cost-to-income ratio -> Cost-Income Ratio

cb -> Corporate Banking
corp bank -> Corporate Banking
corporate bank -> Corporate Banking

wm -> Wealth Management
wealth mgmt -> Wealth Management

pb -> Private Banking
private bank -> Private Banking
```

This map is used for deterministic replacement.

Recipe 12: Applying Canonical Terminology

The harmonizer applies case-insensitive regular-expression matching. Word boundaries are used so that short aliases do not accidentally match inside unrelated words. For example, an alias such as IB should not match the IB characters inside an unrelated word.

The alias itself is escaped before being converted to a regular expression so that characters such as / . ( ) + are interpreted literally.

Recipe 13: Handling Longest-Match Aliases

A key design rule is that more specific aliases must be matched before shorter aliases.

For example, “CB rate” is more specific than “CB.” Therefore aliases are sorted by length before the final alias map is constructed. This prevents a shorter generic alias from modifying a longer and more specific expression incorrectly.

Another important protection is the case-sensitive no-op check. An alias such as “cost-income ratio” must still be recognized as an alias for “Cost-Income Ratio” even though the strings differ only in capitalization. This prevents inner words such as “income” from being substituted independently and producing incorrect results.

Recipe 14: Generating the Harmonization QA Report

The harmonizer produces: outputs/dataset_harmonization_qa_report.txt

The current QA report records:

- Total substitutions made: 4,296

- Unique alias to canonical mappings: 244

The substitutions are sorted by frequency. The most frequent substitutions in the current dataset include:

Alias	Canonical	Occurrences
roe	Return on Equity	323
cir	Cost-Income Ratio	263
transaction banking	Transaction Banking	143
asset management	Asset Management	137
change	Change Analysis	135
retail banking	Retail Banking	127
private banking	Private Banking	125
overall	Global	120
cost	Cost	116
wealth management	Wealth Management	98
revenue	Revenue	88
order	Rank	85
ranking	Rank	57
equity return	Return on Equity	53
nim	Net Interest Margin	48

The QA report is intended for human review. It allows a developer or reviewer to determine whether substitutions look reasonable.

Recipe 15: Validating Harmonization Results

After running:

```
python -c "from src.harmonizer import run_harmonization_raw; run_harmonization_raw()"
```

verify that outputs/Dataset_Harmonized.csv exists. Then:

```
import pandas as pd

df = pd.read_csv("outputs/Dataset_Harmonized.csv")

print(df.shape)
print(df.columns.tolist())
print(df.head())
```

Expected structure: (637, 6) with Conversation_ID, Date, Type, Question_Number, Conversation, and Conversation_harmonized.

Review outputs/dataset_harmonization_qa_report.txt before deploying changes to the dictionaries.

6. Similar Question Search

Recipe 16: Preparing the Search Dataset

The search module loads outputs/Dataset_Harmonized.csv. The rows are deliberately loaded in their original order. The search module does not locally merge questions. The historical dataset is formatted for Gemini using [Conversation_ID Question_Number] followed by the question text. The Question Number is generated deterministically during harmonization.

Recipe 17: Harmonizing a New Question

The evaluator's question is processed using the same dictionaries as the historical dataset.

For example: “What is the CIR for CB in Q1 2026?” can be normalized conceptually toward “What is the Cost-Income Ratio for Corporate Banking in Q1 2026?”

The purpose is to ensure that the new question and historical questions use a shared canonical vocabulary. The system therefore does not require Gemini to independently resolve every abbreviation.

Recipe 18: Preparing the Gemini Prompt

The search module creates a prompt containing:

```
NEW QUESTION:
<new harmonized question>

HISTORICAL DATASET:
[C1 Q1] ...
[C1 Q2] ...
[C2 Q1] ...
...
```

The system instruction is stored separately in instructions/similarity_instructions.txt.

This is important because similarity behavior can be changed without modifying the Python search implementation. The instruction tells Gemini to:

- compare the new question against the historical questions
- evaluate each candidate independently
- consider the underlying financial analysis
- recognize split questions
- distinguish unrelated questions within the same conversation
- scan the entire dataset
- return every genuine match
- provide a similarity score
- provide a short reasoning statement

Recipe 19: Running the Gemini Search

The current implementation performs one evaluator question plus the entire historical dataset in a single Gemini API request.

1. There is no local TF-IDF candidate retrieval stage in the current implementation.
2. There is no local embedding search.
3. There is no graph-based candidate retrieval.
4. There is no separate pairwise verification stage.

Gemini receives the historical question pool and performs the semantic interpretation. The model is google/gemini-3.8-flash through OpenRouter.

Recipe 20: Validating the Gemini Response

The Gemini response is required to conform to a JSON schema.

Each match contains:

- conversation_id
- question_number
- matched_question_text
- similarity_score
- reasoning

The top-level object contains matches. The response parser first attempts direct JSON parsing. If that fails, it attempts to extract a JSON object from surrounding text. If the response appears truncated, a specific error is raised.

This protects the application from malformed or incomplete model responses.

Recipe 21: Ranking and Limiting Results

Similarity scores are expected to be between 0.0 and 1.0. Results are sorted in descending order. The current configuration limits the displayed results to 20. The UI displays the score as a percentage: for example, 0.95 becomes 95%.

The search instruction only asks Gemini to return matches at or above 0.50. The UI therefore describes 50% as the display floor.

Recipe 22: Recovering the Original Historical Question

The model may compare harmonized text. However, the user interface should show the original historical wording. The system therefore uses Conversation_ID and Question_Number to identify the exact historical question.

This provides traceability between the model comparison, the question identifier, the original historical text, and the evaluator UI.

The evaluator can therefore see the actual historical wording rather than only the normalized representation.

7. User Interface

Recipe 23: Starting the Dash Application

Run python app.py. The application starts on http://127.0.0.1:8050. The current UI uses Dash plus Dash Mantine Components.

The interface contains:

- application header
- evaluator identity badge
- light/dark mode switch
- “How it works” section
- example questions
- question input
- Search button
- Clear button
- loading state
- result summary
- result cards
- match-strength labels
- reasoning chips

- error notifications

Recipe 24: Evaluator Identification

When the application loads, the evaluator is asked to provide their name. The name is stored in the Dash session store. The name is attached to every search audit record. The application does not allow a search to proceed without an evaluator identity. The evaluator name is displayed in the interface as: Evaluator: <name>

Recipe 25: Entering a Search Question

The evaluator can type a question in natural language. Abbreviations and informal wording are allowed. The application provides example questions for demonstration. Current examples include:

Example 1: qtrs 25 give me cost and revenue for Wealth Management

Example 2: For Q1 2026, how do CB, Markets, Asset Management, and HNW compare when it comes to equity returns? Also, does the Cost-to-Income ratio for any of these divisions look out of line with where they rank on Return on Equity?

Example 3: Can you compare PB, Transaction and Corporate Banking, Global Banking, and Consumer Banking for 4th Q 25 based on revenue, equity return, and CIR? Which one looks strongest overall?

Recipe 26: Understanding Search Results

When matches are found, the UI displays a result summary (the number of matching historical questions), the best match (the highest similarity score), the average match (the average score across the displayed matches), and the search duration (the time taken to perform the search).

Each individual match card contains:

- match number
- match-strength label
- similarity score
- Conversation ID
- Question Number
- original historical question
- reasoning

Recipe 27: Understanding Match-Strength Labels

The current UI uses these bands:

Score	Label	Meaning
>=95%	Near-identical question	Same divisions, metrics, period and request; only the wording differs

Score	Label	Meaning
>=85%	Same question, minor differences	Same underlying request with one less precise detail
>=70%	Closely related question	Same core request but one factor differs

Scores below 70% receive the fallback label “Weak match.” The configured display floor is 50%; matches below this threshold are not requested from Gemini.

Recipe 28: Handling UI Errors

The UI converts technical errors into user-facing messages. Examples include:

Technical Condition	User-Facing Message
Missing API key	The AI service isn't set up on this computer yet.
Rate limit	The AI service is busy right now.
Missing dataset	The question history couldn't be loaded.
Network failure	The AI service couldn't be reached. Technical details can be expanded by the development team.

8. Gemini and OpenRouter Integration

Recipe 29: Configuring the Gemini Model

The current model setting is: GEMINI_MODEL = "google/gemini-3.8-flash"

The model is accessed through OpenRouter. The application does not call the Gemini service directly.

Recipe 30: OpenRouter Communication

The configured OpenRouter endpoint is: https://openrouter.ai/api/v1

The project uses an OpenAI-compatible interface. The preferred transport is the OpenAI Python package.

A requests implementation is available as a fallback.

The architecture is:

```
Application
↓
GeminiClient
↓
OpenRouter
↓
Google Gemini
```

Recipe 31: Structured JSON Responses

The model response is constrained using a JSON schema. Conceptually:

```
{
  "matches": [
    {
      "conversation_id": "C42",
      "question_number": "Q3",
      "matched_question_text": "Original question text",
      "similarity_score": 0.95,
      "reasoning": "Same division, metric and reporting period."
    }
  ]
}
```

The schema does not allow arbitrary additional properties.

Required fields are:

- conversation_id
- question_number
- matched_question_text
- similarity_score
- reasoning

Recipe 32: Rate Limiting

The current configuration is: GEMINI_RPM = 10

The client uses a simple self-throttling mechanism. The minimum interval is calculated from the configured requests-per-minute value. This prevents rapid bursts of requests.

Recipe 33: Retry and Backoff

The current maximum number of attempts is: GEMINI_MAX_RETRIES = 5

Retry logic is used for errors associated with HTTP 429, rate limits, resource exhaustion, HTTP 503, unavailable services, overload, and high demand.

The retry delay uses exponential backoff with random jitter. Conceptually: attempt 1 fails, a backoff follows, attempt 2 fails, a longer backoff follows, attempt 3 follows, and so on.

Recipe 34: API Error Handling

The client attempts the configured transport.

- If the OpenAI-compatible package cannot be used, the application falls back to requests.
- If all retries fail, the client raises RuntimeError with information about the final failure.

The UI converts the technical error into a user-friendly message.

9. Testing and Quality Assurance

Recipe 35: Running the Test Suite

Run pytest -q.

- The current project contains 23 tests.
- The current supplied project passes all 23.

The test suite covers:

- harmonization
- Gemini response parsing
- retry behavior
- similarity-search result processing

Recipe 36: Harmonizer Tests

The harmonizer tests cover important behavior including:

- alias replacement
- case handling
- word boundaries
- longest alias matching
- duplicate canonical handling
- real dictionary regression cases
- multi-word financial terms
- hyphenated terms

The tests are especially important because harmonization directly changes the text supplied to Gemini.

Recipe 37: Gemini Client Tests

The Gemini tests cover:

- valid JSON
- JSON embedded in surrounding text
- empty match arrays
- malformed JSON
- truncated responses
- successful calls
- retry behavior
- failure after maximum retries

The tests use mocked behavior rather than requiring a live API call. This avoids unnecessary API usage during development and CI.

Recipe 38: Similarity Search Tests

The search tests cover:

- loading the harmonized dataset
- missing input files
- missing instructions
- result sorting
- malformed matches
- score formatting
- conversation ID handling
- question-number handling
- historical question lookup

Recipe 39: Linting

The project uses Ruff.

Run ruff check .

The configured target is Python 3.11. The project focuses linting on rules that identify likely programming problems rather than enforcing purely stylistic formatting.

Recipe 40: Continuous Integration

The project contains a GitHub Actions workflow. CI provides an automated quality gate before deployment. The intended CI process is:

```
Push / Pull Request
↓
Install dependencies
↓
Run Ruff
↓
Run Pytest
↓
Build Docker image where configured
↓
Publish deployment image on the appropriate branch
```

10. Logging and Auditability

Recipe 41: Application Logging

The logging level is configured through LOG_LEVEL.

Supported values:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

The default is INFO.

DEBUG should normally only be used during troubleshooting because it can produce significantly more output.

Recipe 42: Search Audit Logging

Every search is recorded in logs/usage_YYYY-MM-DD.jsonl.

A typical audit record contains:

```
{ "timestamp_utc": "...", "evaluator": "...", "question": "...", "elapsed_seconds": 4.12, "match_count": 2, "matches": [...], "error": null }
```

If a search fails, the error is recorded. Logging failure itself should not prevent the evaluator from receiving search results.

Recipe 43: Reviewing Search Logs

A log file can be inspected with cat logs/usage_2026-09-28.jsonl, or with Python:

```
import json
with open("logs/usage_2026-09-28.jsonl", encoding="utf-8") as f:
    for line in f:
        record = json.loads(line)
        print(record)
```

Audit logs can be used to review:

- who ran a search
- what question was entered
- when the search occurred
- how long the search took
- how many matches were found
- which matches were returned
- whether an error occurred

11. Outputs and Results

Recipe 44: Harmonized Dataset Output

The main generated dataset is outputs/Dataset_Harmonized.csv. It contains:

- Conversation_ID
- Date
- Type
- Question_Number
- Conversation
- Conversation_harmonized

The raw historical text is retained alongside the harmonized version. This allows later verification of substitutions.

Recipe 45: Harmonization QA Output

The QA file is outputs/dataset_harmonization_qa_report.txt. It provides total substitutions, unique alias-to-canonical mappings, and substitution frequency.

The current QA run reports:

1. Total substitutions made: 4,296
2. Unique alias to canonical mappings: 244

Recipe 46: Search Results

Search results are not permanently stored as a separate CSV by the current application. Instead, the result is returned by Gemini, validated, processed by the search module, displayed in the Dash UI, and recorded in the usage log.

This keeps the current application lightweight.

Recipe 47: Result Interpretation

A high similarity score indicates that the model considers the questions substantively similar.

The score should be interpreted together with the historical question, the Conversation ID, the Question Number, the reasoning, the relevant financial division, the metric, the reporting period, and the requested analysis.

The score should not be interpreted as a probability.

A human evaluator remains responsible for determining whether the historical question is suitable for the specific evaluation task.

12. Reproducibility

Recipe 48: Reproducing the Development Environment

Use:

```
git clone <repository-url>
cd model-performance-evaluation-framework-final

python -m venv .venv
```

Activate the environment. Then:

```
pip install -e ".[dev]"
```

Configure .env. Then:

```
pytest -q
```

Recipe 49: Reproducing a Harmonization Run

Run:

```
python -c "from src.harmonizer import run_harmonization_raw; run_harmonization_raw()"
```

Verify outputs/Dataset_Harmonized.csv and outputs/dataset_harmonization_qa_report.txt. The harmonized dataset should have the expected six-column structure.

Recipe 50: Reproducing a Search

1. Start python app.py and open http://127.0.0.1:8050.
2. Enter an evaluator name.
3. Enter a question.
4. Press search.
5. Review the matches, scores, historical questions, reasoning, and search duration.

Then inspect the corresponding logs/usage_YYYY-MM-DD.jsonl.

Recipe 51: Using a New Dataset

A replacement dataset should conform to the columns: Conversation_ID, Date, Type, and Conversation.

Before replacing the dataset:

- Validate the column names.
- Validate the number of rows.
- Validate Type.
- Check missing values.
- Check conversation structure.
- Check that user questions can be identified.
- Run the harmonizer.
- Review the QA report.
- Run the test suite.
- Perform manual search acceptance tests.

The raw source should remain untouched.

13. Deployment

Recipe 52: Building the Docker Image

The Dockerfile uses python:3.12.7-slim. The image installs the production requirements and copies the application.

Build:

```
docker build -t similar-question-search .
```

Recipe 53: Running the Docker Container

Run:

```
docker run \
  -p 8050:8050 \
  -e GEMINI_API_KEY="your_openrouter_key" \
  similar-question-search
```

The application listens on 0.0.0.0:8050 inside the container.

The Dockerfile sets:

```
DASH_HOST=0.0.0.0
DASH_PORT=8050
DASH_NO_BROWSER=1
PYTHONUNBUFFERED=1
```

The application runs as an unprivileged user with UID/GID 1000.

Recipe 54: GitHub Container Registry

The CI/CD process is designed to build and publish the container image to GitHub Container Registry. The deployment flow is:

```
GitHub repository → GitHub Actions → Docker build → GHCR image → Portainer → Running container
```

Recipe 55: Portainer Deployment

The application is designed for the university Data Lab deployment path. The deployment process is:

1. Build the Docker image.
2. Publish the image to GHCR.
3. Configure GHCR credentials in Portainer.
4. Create the application container.
5. Configure environment variables.
6. Set GEMINI_API_KEY.
7. Expose port 8050.
8. Configure persistent storage.
9. Start the container.

10. Verify the health check.
11. Open the application endpoint.

The container does not require a GPU, local model inference, a backend notebook, LocalTunnel, or Cloudflare Tunnel.

The application calls OpenRouter externally.

Recipe 56: Persistent Logs and Outputs

The container creates /app/logs and /app/outputs.

These directories should be persisted through mounted storage when long-term auditability is required. Without persistent storage, container recreation can remove locally stored logs and generated outputs.

Recommended mounts:

- Host volume to /app/logs
- Host volume to /app/outputs

The application should continue to use environment variables for secrets.

14. Troubleshooting

14.1 Installation Problems

Problem	Solution
Python not found	Verify python --version. Install Python 3.11 or newer.
Pip installation fails	Upgrade pip: python -m pip install --upgrade pip. Then reinstall dependencies.

14.2 Dataset Problems

Problem	Solution
Dataset not found	Verify: data/Dataset_Raw.xlsx exists.
Harmonized file missing	Run: python -c "from src.harmonizer import run_harmonization_raw; run_harmonization_raw()"

14.3 Harmonization Problems

Problem	Solution
An alias is not replaced	Check data/harmonization_entities.csv and data/harmonization_time_formats.csv and confirm that the alias and the canonical value exist, the type is not excluded, and the alias is spelled correctly.

14.4 Gemini/API Problems

Problem	Solution
Missing API key	Check GEMINI_API_KEY
HTTP 429	The API rate limit has been reached. Wait and retry. The application also performs retry/backoff automatically.
HTTP 503	The upstream service may be unavailable or overloaded. Retry later.
Model not found	Check GEMINI_MODEL in config/settings.py

14.5 UI Problems

Problem	Solution
Blank page	Verify the installed Dash version. The project requires dash >= 4.0.0 and dash-mantine-components >= 2.0.0.
Port 8050 already in use	Change DASH_PORT

14.6 Docker Problems

Problem	Solution
Container starts but cannot be reached	Verify -p 8050:8050 and DASH_HOST=0.0.0.0
Permission errors in logs or outputs	The container uses UID/GID 1000. Ensure mounted directories are writable by that user.
Health check fails	Verify that the Dash application is listening on 127.0.0.1:8050 inside the container.

14.7 Logging Problems

If logs are missing:

1. Check the logs/ directory.
2. Check filesystem permissions.
3. Check LOG_LEVEL.
4. Check whether the container volume is mounted.
5. Check application startup logs.

15. Best Practices

15.1 Code Organization

- Use app.py for UI behavior.
- Use src/harmonizer/ for deterministic terminology normalization.
- Use src/search/ for search orchestration.
- Use src/llm/ for model communication.
- Use src/usage_log.py for audit logging.
- Do not duplicate configuration values across modules.

15.2 Data Management

- Never modify Dataset_Raw.xlsx as part of processing.
- Generated files should be written to outputs/
- Always review the harmonization QA report after dictionary changes.

15.3 Harmonization

- Use the dictionaries instead of hardcoding replacements.
- Prefer alias to canonical mappings rather than adding one-off transformations inside Python.
- Always run tests after changing dictionaries.
- Review the most frequent substitutions.

15.4 LLM Usage

- Keep the model name configurable.
- Keep the system instruction in instructions/similarity_instructions.txt.
- Do not hardcode large instruction prompts into Python.
- Validate all model responses.
- Do not assume that a model response is correct simply because it is valid JSON.

15.5 Security

- Never commit .env
- Never expose GEMINI_API_KEY
- Never store secrets in Docker images.
- Use Portainer environment variables for deployment secrets.
- Avoid putting sensitive search information into debug logs.

15.6 Testing

- Run pytest -q before deployment.
- Run ruff check . before submitting code changes.
- Do not require live Gemini calls for unit tests.
- Use mocked model behavior wherever possible.

15.7 Reproducibility

- Record the software version, Python version, model name, configuration, dataset version, dictionary version, and instruction version.
- Keep generated outputs separate from raw inputs.

15.8 Deployment

- Use the Docker image rather than manually configuring production Python environments.
- Use persistent storage for logs/ and outputs/
- Supply API keys through environment variables.
- Use the container health check to verify availability.

16. Complete Example

16.1 Example Input

Consider the evaluator question: “what's the CIR for corporate banking in the second quarter of 2025?”

The evaluator enters the question into the Similar Question Search interface.

16.2 Harmonization

The question contains several expressions that can be normalized. Conceptually:

- CIR becomes Cost-Income Ratio
- corporate banking becomes Corporate Banking

- second quarter 2025 becomes Q2 2025

The resulting harmonized question is compared using the same vocabulary as the historical dataset.

16.3 Search Execution

The application creates a prompt containing:

```
NEW QUESTION:
<harmonized question>

HISTORICAL DATASET:
[C1 Q1] ...
[C1 Q2] ...
[C1 Q3] ...
...
```

The entire historical dataset is supplied in the same model request.

16.4 Gemini Response

Gemini returns structured data conceptually similar to:

```
{
  "matches": [
    {
      "conversation_id": "C34",
      "question_number": "Q1",
      "matched_question_text": "cost income ratio for Corporate Banking Q2 25 pls",
      "similarity_score": 0.95,
      "reasoning": "Same division, same metric and same reporting period."
    }
  ]
}
```

The actual response depends on the model's evaluation of the historical dataset.

16.5 Result Processing

The search module parses the JSON, obtains the matches, validates the returned fields, sorts the matches by similarity, limits the displayed result count, converts scores into percentages for the UI, and identifies the corresponding historical question.

16.6 UI Output

The evaluator sees information such as: 1 earlier question asks the same thing.

Best match: Near-identical question, 95%

Conversation: C34, Question: Q1

cost income ratio for Corporate Banking Q2 25 pls

Why it matches:

- Same division
- Same metric
- Same reporting period

16.7 Audit Record

The search is written to the daily JSONL log. Conceptually:

{ "timestamp_utc": "2026-09-28T...", "evaluator": "Evaluator Name", "question": "what's the CIR for corporate banking in the second quarter of 2025?", "elapsed_seconds": 4.2, "match_count": 1, "matches": [{ "conversation_id": "C34", "question_number": "Q1", "similarity_score": 0.95 }], "error": null }

17. Project Evolution

The project was developed incrementally through eight historical iterations. Each iteration introduced changes to the system's purpose, architecture, user interface, matching strategy, evaluation process, or operational reliability.

The development began as a basic Similar Question Search concept and gradually expanded into a broader Model Performance Evaluation Framework. The later iterations introduced a more structured duplicate-detection and evaluation pipeline. The current system subsequently simplified the operational workflow into a focused Similar Question Search component for evaluators.

The eight historical project iterations are:

1. Similar Question Search, 1st Iteration
2. Similar Question Search, 2nd Iteration
3. Model Performance Evaluation Framework, 3rd Iteration
4. Question Similarity, 4th Iteration
5. Question Similarity Dash Mantine, 5th Iteration
6. Model Performance Evaluation Framework, 6th Iteration
7. Model Performance Evaluation Framework Final, 7th Iteration
8. Model Performance Evaluation Framework Final, 8th Iteration

These iterations represent the historical development path of the project. The current Similar Question Search component should not be confused with the earlier duplicate-detection architecture.

17.1 First Iteration: Similar Question Search

The first iteration established the fundamental concept of the project: allowing an evaluator to enter a financial question and identify historically similar questions.

The initial design was based on a simple processing chain: an evaluator question was compared against the historical dataset to produce a similarity evaluation, yielding similar historical questions.

The main purpose was to determine whether a newly entered financial question had already been asked previously, even when the wording was not identical. The system therefore focused on semantic rather than exact textual matching.

At this stage, the interface was implemented as a simpler Dash application. The primary objective was to establish the core functionality and demonstrate that an LLM could compare a new financial question with historical questions.

The first iteration therefore served as the proof-of-concept stage for the Similar Question Search functionality.

17.2 Second Iteration: Similar Question Search

The second iteration continued development of the Similar Question Search concept.

The fundamental architecture remained: an evaluator question was compared against the harmonized historical dataset through Gemini to produce similar historical questions.

The system continued to focus on semantic matching between a newly entered evaluator question and previously stored financial questions.

The second iteration refined the initial concept and provided a stronger foundation for subsequent development.

At this stage, the project was still primarily a question-search solution rather than a complete model-performance evaluation framework.

17.3 Third Iteration: Model Performance Evaluation Framework

The third iteration expanded the scope of the project from a basic Similar Question Search tool into a broader Model Performance Evaluation Framework.

The project began addressing the larger problem of systematically evaluating financial questions and identifying questions that represented the same underlying information.

This change introduced a stronger emphasis on:

- Structured dataset processing
- Financial terminology normalization
- Question similarity
- Evaluation methodology
- Repeatable processing

- Framework-level organization

The project therefore moved beyond a simple user-facing search concept and toward a reusable evaluation architecture.

17.4 Fourth Iteration: Question Similarity

The fourth iteration concentrated on improving the question-similarity functionality. The objective was to improve the identification of questions that were semantically equivalent despite differences in wording.

The matching process considered the financial context of the questions rather than relying only on exact textual overlap. This iteration helped establish question similarity as a central capability of the overall framework.

The project continued moving toward a system capable of recognizing variations in:

- Financial terminology
- Entity names
- Metrics
- Reporting periods
- Question wording

17.5 Fifth Iteration: Question Similarity with Dash Mantine

The fifth iteration introduced the Dash Mantine interface and represented a significant improvement in the user experience. The interface was redesigned to make the system more suitable for non-technical users, demonstrations, and evaluator workflows.

The improved interface introduced several usability features, including:

- A How It Works section
- Example questions
- Plain-language match-strength labels
- Reasoning chips
- Friendly error messages
- Evaluator attribution

These changes made the similarity results easier to interpret. Instead of presenting only technical similarity information, the system began communicating results in a more understandable form.

The fifth iteration therefore improved the project from both a technical and usability perspective.

17.6 Sixth Iteration: Model Performance Evaluation Framework

The sixth iteration expanded the project into a more comprehensive duplicate-detection and model-evaluation framework.

The historical architecture developed into a staged pipeline: harmonization, candidate retrieval, fingerprint and entity checks, Gemini verification, report cross-check, clustering, and evaluation.

This architecture introduced multiple retrieval strategies for identifying candidate matches. The historical framework investigated:

- TF-IDF retrieval
- Finance-specific embedding retrieval
- Graph similarity

The purpose of using multiple retrieval methods was to compare different approaches for identifying potentially equivalent financial questions. The framework also introduced deterministic verification before LLM verification. This allowed inexpensive and obvious matches to be resolved before involving Gemini.

Financial report information was also incorporated into the verification process so that similarity decisions were not based solely on question wording.

17.7 Seventh Iteration: Model Performance Evaluation Framework Final

The seventh iteration further refined the Model Performance Evaluation Framework. The project became more modular and configurable, with separate components for:

- Harmonization
- Retrieval
- Financial report parsing
- Fingerprinting
- Gemini verification
- Comparison
- Clustering
- Evaluation

The retrieval layer supported interchangeable retrieval methods, allowing different approaches to be evaluated on the same dataset. The Gemini verification stage was also structured to operate only on candidate pairs that could not already be resolved through deterministic checks.

The framework incorporated batching, rate limiting, retry and backoff mechanisms to make Gemini usage more controlled and reliable. The project also introduced structured output files and evaluation procedures so that results could be compared across retrieval methods.

This iteration therefore focused on improving reproducibility, modularity, reliability and evaluation quality.

17.8 Eighth Iteration: Model Performance Evaluation Framework Final

The eighth iteration represents the later finalized stage of the historical Model Performance Evaluation Framework.

The framework provided a complete staged duplicate-detection workflow:

Stage 0, Harmonization
Financial terminology, entities and time expressions were normalized.
↓
Stage 1, Candidate Retrieval
Potentially similar question pairs were identified using the configured retrieval method.
↓
Stage 1B / 1C, Deterministic Verification
Financial fingerprints and entity information were used to resolve cases without unnecessary LLM calls.
↓
Stage 2, Gemini Verification
Remaining ambiguous candidate pairs were evaluated semantically.
↓
Stage 2B, Report Cross-Check
Gemini-positive matches were checked against financial report information.
↓
Stage 3, Clustering
Confirmed relationships were converted into duplicate groups.
↓
Evaluation

The resulting groups were compared against the available ground truth.

The historical framework evaluated TF-IDF, Graph and Embeddings retrieval approaches. The documented evaluation reported 90 of 90 correct groups for all three methods, with different mean question-similarity values.

The historical framework should be understood as an important stage in the project's development rather than as the exact runtime architecture of the current Similar Question Search application.

17.9 Transition to the Current Similar Question Search System

After the historical framework development, the project was refined into the current Similar Question Search component.

The current system focuses on the practical task of allowing an evaluator to enter a new financial question and retrieve semantically similar historical questions.

The current operational workflow is:

```
Historical Dataset
↓
Financial Terminology Harmonization
↓
Evaluator Enters New Question
↓
New Question Harmonization
↓
Gemini Semantic Search
↓
Structured Response Validation
↓
Similarity Ranking
↓
Results Displayed in Dash
↓
Audit Logging
```

This represents a simplification of the earlier duplicate-detection architecture.

The current application is therefore focused on user-facing Similar Question Search, while the earlier multi-stage retrieval, duplicate-detection and clustering architecture remains part of the project's historical development and evaluation work.

17.10 Evolution of the User Interface

The user interface also evolved throughout the project.

Early Stage: the project initially used a simpler Dash interface focused primarily on entering a question and displaying similar results.

Intermediate Stage: the interface became more user-friendly with the introduction of Dash Mantine. The improved interface added:

- How It Works information
- Example questions
- Match-strength labels
- Reasoning information
- Friendly error messages
- Evaluator identification

Current Stage: the current interface provides an evaluator-oriented Similar Question Search workflow. The evaluator identifies themselves, enters a new question, submits the question for search, receives ranked historical matches, reviews similarity scores and explanations, and uses the results to understand whether a similar question already exists.

17.11 Evolution of the Matching Strategy

The matching strategy also changed significantly across the project.

Early Development: the project initially focused on direct semantic comparison between the evaluator's question and historical questions.

Framework Development: the historical framework introduced multiple retrieval strategies, namely TF-IDF, finance-specific embeddings, and graph similarity. Candidate retrieval was followed by deterministic checks and Gemini verification.

Current System: the current Similar Question Search system uses the harmonized historical dataset and Gemini-based semantic search to identify relevant historical questions.

This separation allows the historical evaluation framework and the current user-facing search component to be understood as different stages of the overall project evolution.

17.12 Evolution of Reliability and Quality Assurance

As the project matured, reliability mechanisms were added to support practical use.

These include:

- JSON response validation
- Gemini rate limiting
- Retry and exponential backoff
- Audit logging
- Unit testing
- Continuous integration checks
- Error handling
- Structured outputs
- Docker-based deployment

The current cookbook documents these capabilities as part of the operational system.

These additions changed the project from a prototype into a more maintainable and operationally reliable application.

17.13 Overall Project Evolution

The overall project evolution can be summarized as a progression through six stages.

Prototype: simple Similar Question Search
↓
Semantic Matching: improved financial question similarity
↓
Framework Expansion: Model Performance Evaluation Framework
↓
User Experience: Dash Mantine interface and evaluator-focused features
↓
Advanced Evaluation: multiple retrieval methods, deterministic checks, Gemini verification and report cross-checking
↓
Reliability: testing, logging, rate limiting, retry mechanisms and deployment support
↓
Current System: simplified and operational Similar Question Search application

The project therefore evolved from a proof-of-concept semantic search application into a broader evaluation framework and subsequently into a focused operational Similar Question Search component.

This evolution demonstrates a progression from experimentation toward structured evaluation, maintainability, usability and operational deployment.

18. Appendix

A. Installation Commands

```
git clone <repository-url>
cd model-performance-evaluation-framework-final
python -m venv .venv

Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate

Install: pip install -e ".[dev]"

Configure: cp .env.example .env

Run tests: pytest -q

Run linting: ruff check .

Run harmonization: python -c "from src.harmonizer import run_harmonization_raw;
run_harmonization_raw()"

Start application: python app.py
```

B. Main Configuration Values

Setting	Value
GEMINI_MODEL	google/gemini-3.8-flash
GEMINI_RPM	10
GEMINI_MAX_RETRIES	5
MAX_RESULTS_DISPLAYED	20
MATCH_DISPLAY_FLOOR	0.50
LOGS_DIR	logs
RAW_DATASET_PATH	data/Dataset_Raw.xlsx
RAW_DATASET_OUTPUT_PATH	outputs/Dataset_Harmonized.csv
RAW_HARMONIZATION_REPORT	outputs/dataset_harmonization_qa_report.txt

C. Match-Strength Bands

Score	Label
>= 95	Near-identical question
>= 85	Same question, minor differences
>= 70	Closely related question
< 70	Weak match (fallback label)
50	Display floor

D. Dataset Schema

Raw Dataset:

Column	Type/Meaning
Conversation_ID	Conversation identifier
Date	Date
Type	user or model
Conversation	Full turn text

Harmonized Dataset:

Column	Meaning
Conversation_ID	Conversation identifier
Date	Date
Type	user or model
Question_Number	Q1, Q2, Q3, etc. for user questions
Conversation	Full turn text
Conversation_harmonized	Text after terminology normalization

E. Harmonization Dictionary Schema

The entity dictionary contains: Type, Key, Canonical, Alias1, Alias2, through Alias18.

The time dictionary contains: Type, Key, Canonical, Alias1, Alias2, through Alias16.

The harmonizer automatically detects columns beginning with “Alias” so additional alias columns can be supported without changing the core alias-discovery logic.

F. Gemini Response Schema

```
{
  "matches": []
}
```

A match object contains:

```
{
  "conversation_id": "C42",
  "question_number": "Q3",
  "matched_question_text": "Exact historical question",
  "similarity_score": 0.95,
  "reasoning": "Short explanation"
}
```

G. Known Acceptance Test Cases

Test Case 1

Input: For Q1 2026, how do CB, Markets, Asset Management, and HNW compare when it comes to equity returns? Also, does the Cost-to-Income ratio for any of these divisions look out of line with where they rank on Return on Equity?

The historical project documentation identifies C4, C34, and C64 as the expected matching conversations for this test case.

Test Case 2

Input: Can you compare PB, Transaction and Corporate Banking, Global Banking, and Consumer Banking for 4th Q 25 based on revenue, equity return, and CIR? Which one looks strongest overall?

The historical project documentation identifies C9, C39, and C69 as the expected matching conversations for this test case.

These acceptance cases should be used after installation to confirm that the application is operating correctly against the supplied historical dataset.

H. Docker Configuration

The Docker image: python:3.12.7-slim

The application runs as a non-root user.

The exposed port is: 8050

The container environment includes:

```
DASH_HOST=0.0.0.0
DASH_PORT=8050
DASH_NO_BROWSER=1
PYTHONUNBUFFERED=1
```

I. Error Reference

Error	Meaning	Action
Missing API key	OpenRouter key unavailable	Configure GEMINI_API_KEY
401	Authentication failure	Verify API key
429	Rate limit	Wait and retry
503	Service unavailable	Retry later
Model not found	Invalid model configuration	Check GEMINI_MODEL
Dataset not found	Input file missing	Check data/
Harmonized dataset missing	Harmonization not run	Run harmonizer
Instructions missing	Similarity instructions unavailable	Check instructions/
JSON parsing error	Invalid model response	Inspect Gemini response
Truncated response	Model output incomplete	Retry / review token limit
Port in use	Port 8050 occupied	Change DASH_PORT

J. Glossary

Term	Definition
Alias	An alternative expression, abbreviation, spelling, or wording that maps to a canonical term.
Canonical Term	The standardized terminology used by the harmonization system.
Harmonization	The deterministic process of converting aliases into canonical financial terminology.
Conversation ID	Identifier associated with a historical conversation.
Question Number	Identifier for an individual user question within a conversation.
Split Question	A question represented by two consecutive user rows with the same Conversation ID and Question Number.
Similarity Score	A model-generated value from 0.0 to 1.0 representing the assessed semantic similarity between the new question and a historical question.
Gemini	The Google generative AI model used by the project for semantic question comparison.
OpenRouter	The API gateway through which the application accesses the Gemini model.
JSON Schema	A formal definition of the expected structure of the Gemini response.
Rate Limiting	Controlling the frequency of API requests.
Retry / Backoff	Waiting and retrying after recoverable API failures.
Audit Log	A JSONL record of searches performed by evaluators.
QA Report	The harmonization report showing the substitutions performed by the terminology pipeline.

K. Known Limitations

The current implementation has several limitations that should be understood before production-scale expansion.

Full-dataset model request: each search sends the entire historical question dataset to Gemini in one API request. This keeps the architecture simple and minimizes the number of requests, but the approach may become less suitable as the historical dataset grows significantly.

LLM-dependent semantic interpretation: the final semantic matching decision is made by Gemini. Therefore results may vary with model behavior or changes to the instruction prompt.

Model-generated similarity scores: similarity scores are model judgments rather than statistically calibrated probabilities.

No dedicated local retrieval stage: the current implementation does not use a separate TF-IDF, embedding, or graph candidate-retrieval layer.

Dataset-specific harmonization: the dictionaries were created for the financial terminology represented in the project dataset. A substantially different financial domain may require additional aliases and canonical terms.

API dependency: live search requires access to OpenRouter and the configured Gemini model.

Historical question quality: search quality depends partly on the quality and completeness of the historical dataset.

L. References

Project Files:

- README.md
- app.py
- config/settings.py
- src/harmonizer/harmonizer.py
- src/harmonizer/__init__.py
- src/search/similarity_search.py
- src/llm/gemini_client.py
- src/usage_log.py
- src/logging_config.py
- instructions/similarity_instructions.txt
- tests/test_harmonizer.py
- tests/test_gemini_client.py
- tests/test_similarity_search.py
- Dockerfile
- pyproject.toml
- requirements.txt

Data Files:

- data/Dataset_Raw.xlsx
- data/harmonization_entities.csv
- data/harmonization_time_formats.csv

Generated Files:

- outputs/Dataset_Harmonized.csv

- outputs/dataset_harmonization_qa_report.txt
- logs/usage_YYYY-MM-DD.jsonl

Historical Documentation:

- Cookbook_Model_Performance_Evaluation_Framework